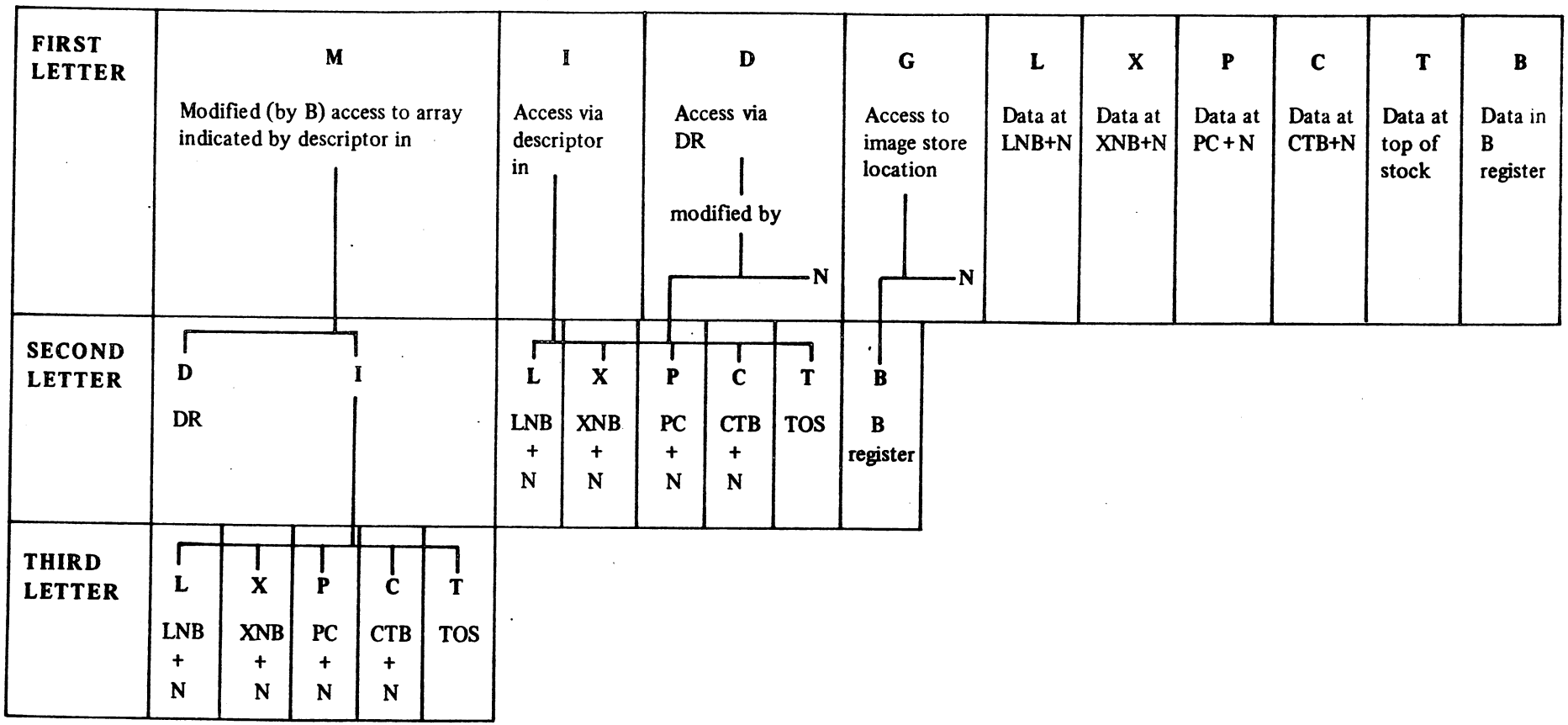
North Staffs Polytechnic SFL handout
circa. 1982

PERMITTED ARITHMETIC OPERATIONS

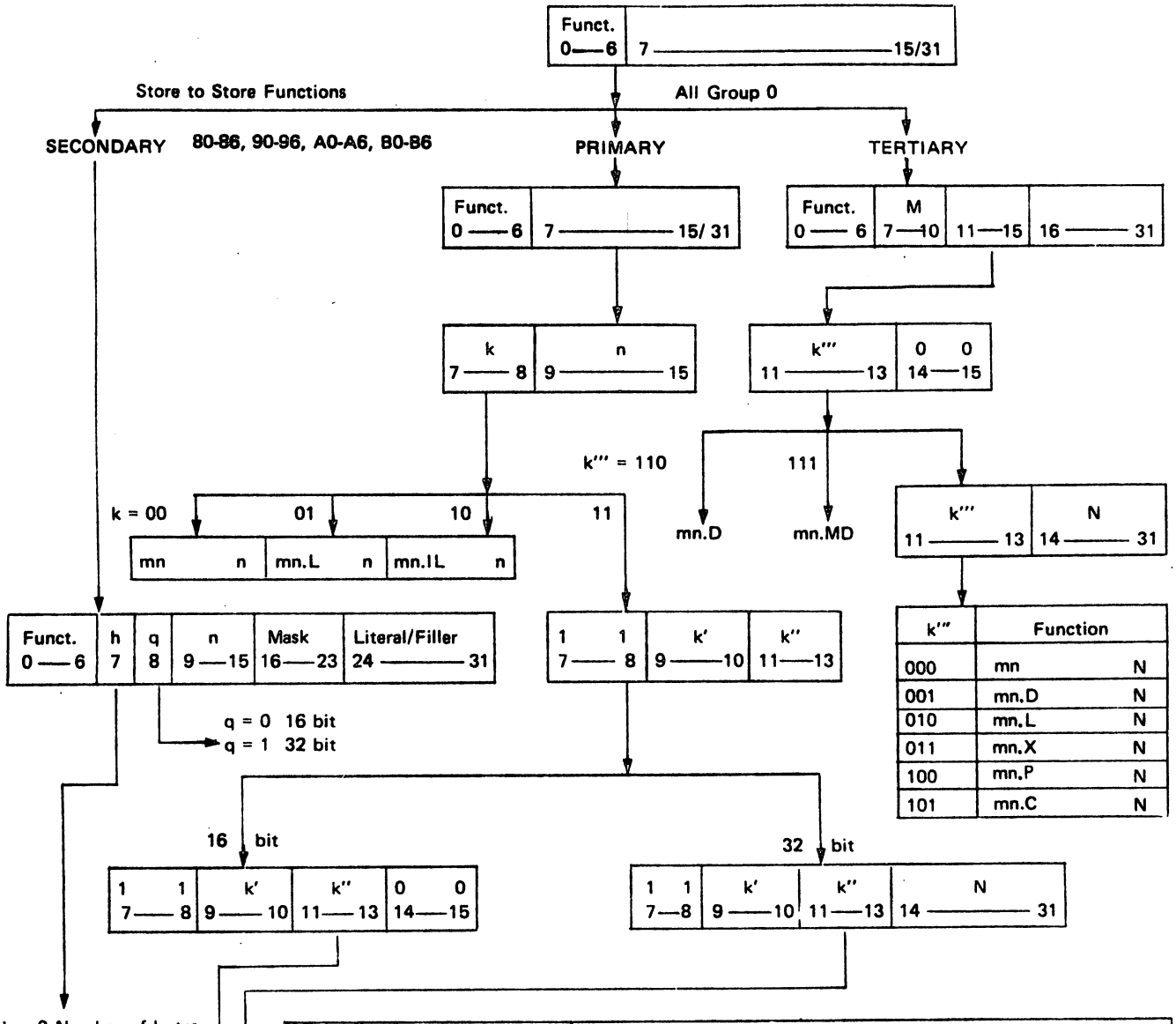
The table shows the final accumulator size (S,D,Q) for each valid combination of function type, data type, and accumulator size.

DATA TYPE \ FUNCTION TYPE / ACS		AD SB RSB	MY DV RDV	MYD	DVD	MDV	CP	SH	SC	AND OR NEQ	ROT SHS SHZ
INT (I)	SINGLE DOUBLE QUAD	S D	S D	D		S D	S D	S D			
WORD (U)	SINGLE DOUBLE QUAD	S					S D	S D		S D	S D
REAL (R)	SINGLE DOUBLE QUAD	S D Q	S D Q	D Q	S D		S D Q		S D Q		
DECIMAL (D)	SINGLE DOUBLE QUAD	S D Q	S D Q	D Q		S D Q	S D Q	S D Q			

INTERPRETING SFL VARIANTS



FUNCTION DECODE



k' / k''	DIRECT	INDIRECT		
	0	1 Desc. in DR modified	2 Desc. in store	3 Desc. in store modified by B
0	mn N	mn.D N	mn.G N	mn.GB
1	UNASSIGNED			
2	mn.L N	mn.DL N	mn.IL N	mn.MIL N
3	mn.X N	mn.DX N	mn.IX N	mn.MIX N
4	mn.P N	mn.DP N	mn.IP N	mn.MIP N
5	mn.C N	mn.DC N	mn.IC N	mn.MIC N
6	mn.T	mn.DT	mn.IT	mn.MIT
7	mn.B	Unassigned	mn.D	mn.MD

2900 BASIC FUNCTION CODES

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		10 VAL	20 ADB	30 LCT	40 SL	50 SLD	60 L	70 LDRL	80 TCH	90 PK	A0 SWEQ	B0 MVL	C0 UAD	D0 DAD	E0 IAD	F0 RAD
2	02 JCC	12 CYD	22 SBB	32 MPSR	42 SLSS	52 SLB	62 LSS	72 LDA	82 ANDS	92 INS	A2 SWNE	B2 MV	C2 USB	D2 DSB	E2 ISB	F2 RSB
4	04 JAT	14 INCA	24 DEBJ	34 CPSR	44 SLSD	54 TDEC	64 LSD	74 LDTB	84 ORS	94 SUPK	A4 CPS	B4 CHOV	C4 URSB	D4 DRSB	E4 IRSB	F4 RRSB
6	06 JAF	16 MODD	26 CPB	36 STCT	46 SLSQ	56 INCT	66 LSQ	76 LDB	86 NEQS		A6 TTR		C6 UCP	D6 DCP	E6 ICP	F6 RCP
8		18 PRCL	28 SIG	38 EXIT	48 ST	58 STD	68 RRTC	78 LD	88 EXPA	98 COMA	A8 FLT	B8 FIX	C8 USH	D8 DSH	E8 ISH	F8 RSC
A		1A J	2A MYB	3A ESEX	4A STUH	5A STB	6A LUH	7A LB	8A AND	9A DDV	AA IDV	BA RDV	CA ROT	DA DMY	EA IMY	FA RMY
C		1C JLK	2C VMY	3C OUT	4C STXN	5C STLN	6C RALN	7C LLN	8C OR	9C DRDV	AC IRDV	BC RRDV	CC SHS	DC DMYD	EC IMYD	FC RMYD
E		1E CALL	2E CPIB	3E ACT	4E IDLE	5E STSF	6E ASF	7E LXN	8E NEQ	9E DMDV	AE IMDV	BE RDVD	CE SHZ	DE CBIN	EE CDEC	

JUMPS

J	1A	P	Jump unconditionally. PC' = PC + LIT or OP_{0-30}
DEBJ	24	P	Decrement B & jump if non-zero. B' = B-1, jump if B' ≠ 0.
JCC	02	T	Jump on CC. PC' = PC + LIT or OP_{0-30} if bit (CC) of Mask = 1

Mask	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Jumps if CC is	-	3	2	2 3	1	1 3	1 2	1 2 3	0	0 3	0 2	0 2 3	0 1	0 1 3	0 1 2	A N Y

JAT	04	T	Jump on arithmetic true, jump if 'M' is true.
JAF	06	T	Jump on arithmetic false. jump if 'M' is false.

Where with floating M = 0 ACC = 0; M = 1 ACC > 0; M = 2 ACC < 0
with fixed M = 4 ACC = 0; M = 5 ACC > 0; M = 6 ACC < 0
with decimal M = 8 ACC = 0; M = 9 ACC > 0; M = A, ACC < 0
with B M = C, B = 0; M = D, B > 0; M = E, B < 0
and M = B, DR BOUND = 0; M = F, OV = 1

JLK	1C	P	Jump & link. TOS' = PC, PC' + LIT or OP_{0-30}
CALL	1E	P	Procedure call. (LNB+1)' = descriptor of PC and jump
EXIT	38	P	Procedure exit. stack returned to status quo, jump to link
ESEX	3A	P	Escape exit. PC' = TOS, D' = 1 with descriptor in DR

See separate list for SFL pseudo-tertiary jumps.

DR INSTRUCTIONS

LD	78	P	Load DR. DR' = OP_{0-63}, CC' = descriptor type
LDRL	70	P	Load relative. DR' = OP_{0-63} + OP address, CC' = descriptor type
LDA	72	P	Load address, $DR'_{32-63} = OP_{0-31}$
LDTB	74	P	Load type and bound. $DR'_{0-31} = OP_{0-31}$
LDB	76	P	Load bound. $DR'_{8-31} = OP_{8-31}$
SLD	50	P	Stack & load DR. TOS' = DR, DR' = OP_{0-63}, CC' = descriptor type
STD	58	P	Store DR. OP'_{0-63} = DR
INCA	14	P	Increment address. $DR'_{32-63} = DR_{32-63} + OP_{0-31}$
MODD	16	P	Modify DR. $DR'_{8-31} = DR_{8-31} - OP_{8-31}$, $DR'_{32-63} = DR_{32-63} + (OP_{0-31}$ scaled)
VAL	10	P	Validate address. check validity of DR by OP_{8-11} as ACR

OK CC' = 0, | read only CC' = 1, | write only CC' = 2, | invalid CC' = 3.

SIG	28	P	Start significance. If CC' = 0, OP_{0-63} = type 1 descriptor of length = 1.

address = address - 1 and CC set to 1. If CC ≠ 0, do nothing.

B INSTRUCTIONS

LB	7A	P	Load B. B' = OP_{0-31}, OV' = 0
SLB	52	P	Stack & Load B. TOS' = B, B' = OP_{0-31}, OV' = 0
STB	5A	P	Store B. OP'_{0-31} = B, OV' = 0

B INSTRUCTIONS (Cont.)

ADB	20	P	Add to B. B′ = B + OP_{0-31}, OV′ overflow
SBB	22	P	Subtract from B. B′ = B - OP_{0-31}, OV′ = overflow
MYB	2A	P	Multiply B. B′ = B. OP_{0-31}, OV′ = overflow
CPB	26	P	Compare B. B = OP_{0-31}, CC′ = 0, | B < OP, CC′ = 1. | B > OP, CC′ = 2 |
CPIB	2E	P	Compare B increment B. as CPB & B′ = B + 1
VMY	2C	P	Dope vector multiply. B′ = (OP_{0-31} - x). y, check $0 \leqslant B' < z$ & $0 < (OP - x) < 2^{31}$; DR points at dope vector containing x,y,z.

STACK CONTROL

LLN	7C	P	Load LNB. LNB′ = OP_{14-29}, OP_{0-13} must equal SSN
STLN	5C	P	Store LNB. OP'_{0-31} = SSN/LNB
RALN	6C	P	Raise LNB. LNB′ = SF - OP_{0-31}
ASF	6E	P	Adjust SF.SF′ = SF + OP_{0-31}, OP may be -ve
PRCL	18	P	Precall. Align SF odd, STLN.T , ASF OP_{0-31}
LXN	7E	P	Load XNB.XNB′ = OP_{0-29}
STXN	4C	P	Store XNB. OP'_{0-31} = XNB
STSF	5E	P	Store SF. OP'_{0-31} = SSN/SF
LCT	30	P	Load CTB. CTB′ = OP_{0-29}
STCT	36	P	Store CTB. OP'_{0-31} = CTB

SUNDRIES

RRTC	68	P	Read real time clock. ~~OP'_{0-63} = clock output.~~ sticks it in LACC.
OUT	3C	P	Out. Cause class 9 interrupt with OP_{0-31} as parameter
IDLE	4E	P	Idle. Suspend instruction sequencing until interrupt occurs
ACT	3E	P	Activate. OP_{0-63} = LSTB; OP_{64-95} is spare; OP_{96-110} = SSN. Load CPU registers with contents of SSN + 1 and start process.

STRING INSTRUCTIONS

SWEQ	A0	S	Scan while equal. LIT ≢ DR stg, not found CC′ = 0 | DR stg > LIT, CC′ = 2 | DR stg < LIT, CC′ = 3
SWNE	A2	S	Scan while unequal. LIT ≡ DR stg, not found CC′ = 0 else 1
CPS	A4	S	Compare strings. DR stg = ACC stg, CC′ = 0 | DR stg > ACC stg, CC′ = 2 | DR stg < ACC stg, CC = 3 |
MV	B2	S	Move. DR string = ACC string
CHOV	B4	S	Check overlap. none, CC′ = 0 | ACC stg > DR stg, CC′ = 1 | ACC stg < DR stg, CC′ = 2 |
MVL	B0	S	Move literal. DR string = literal
TCH	80	S	Table check. check DR stgs with bit in ACC stg and stop if bit = 1 setting CC′=1
TTR	A6	S	Table translate. DR string is translated by ACC stg.
PK	90	S	Pack. ACC = DR string, packed. OV = 0
SUPK	94	S	Suppress & unpack. DR string = ACC unpacked, CC is set
INS	92	S	Conditional insrt. DR string = LIT (CC = 0) or mask (CC ≠ 0)
ANDS	82	S	And string. DR′ stg. = DR stg. & ACC stg. or literal
ORS	84	S	Or string. DR′ stg = DR stg. v ACC stg. or literal
NEQS	86	S	Not equivalent string. DR′ stg = DR stg ≢ ACC stg. or literal.

SEMAPHORE INSTRUCTIONS

INCT	56	P	Increment and test OP' = OP + 1
TDEC	54	P	Test and decrement OP' = OP - 1 OP = 0, CC = 0 | OP 0, CC = 1 | OP < - 1, CC = 2 | OP = - 1, CC = 3 | (OP = final value for INCT and original value for TDEC).

ACC INSTRUCTIONS

L	60	P	Load. ACC' = OP_{ACS}, OV' = 0
LSS	62	P	Set ACS 32 & load. ACS' = 32, ACC' = OP_{0-31}, OV' = 0
LSD	64	P	Set ACS 64 & load. ACS' = 64, ACC' = OP_{0-63}, OV' = 0
LSQ	66	P	Set ACS 128 & load. ACS' = 128, ACC' = OP_{0-127}, OV' = 0
LUH	6A	P	Load upper half. ACS' = 2ACS, ms ½ of ACC' = OP_{ACS}, OV' = 0
CYD	12	P	Copy DR. ACS' = 64 ACC' = DR, OV' = 0
SL	40	P	Stack & load. TOS'_{ACS} = ACC, ACC' = OP_{ACS}, OV' = 0
SLSS	42	P	Stack, set ACS 32 & Load. TOS_{ACS} = ACC, ACS' = 32, ACC = OP_{0-31}, OV' = 0
SLSD	44	P	Stack, set ACS 64. & Load. TOS_{ACS} = ACC, ACS' = 64, ACC = OP_{0-63}, OV' = 0
SLSQ	46	P	Stack, set ACS 128 & Load. TOS_{ACS} = ACC, ACS' = 128, ACC = OP_{0-127}, OV' = 0
ST	48	P	Store. OP_{ACS} = ACC, OV' = 0
STUH	4A	P	Store upper half. $OP'_{½ACS}$ = ms ½ of ACC, ACS' = ½ ACS. OV' = 0
MPSR	32	P	Modify PSR. If OP_{27} = 1, ACS' = OP_{30-31} | If OP_{26} = 1, CC' = OP_{28-29} | If OP_{24} = 1, PM' = 1's at OP_{16-23} = 1's or OP_{25} = 1, PM' = 0's at OP_{16-23} = 0's.
CPSR	34	P	Copy PSR. OP_{0-15} = 0 | OP_{16-23} = PM' | OP_{24-27} = 1110 | OP_{28-29} = CC' | OP_{30-31} = ACS'.

LOGICAL (WORD)

UAD	C0	P	Logical add. ACC' = ACC + OP_{0-31}, CC' = 1 if carry, OV' = 0
USB	C2	P	Logical subtract. ACC' = ACC - OP_{0-31}, CC' = 1 if borrow, OV' = 0
URSB	C4	P	Logical reverse subtract. ACC' = OP_{0-31} - ACC, CC' = 1 if borrow, OV' = 0
UCP	C6	P	Logical compare. ACC = OP_{ACS}, CC' = 0 | ACC < OP, CC' = 1 | ACC > OP, CC = 2 |
USH	C8	P	Logical shift. Shift ACC left OP_{25-31} places or right if OP-ve, OV' = 0
ROT	CA	P	Rotate. Rotate ACC left OP_{26-31} (ACS double) or OP_{27-31} (ACS single) OV' = 0
SHS	CC	P	Shift 32 bits. Shift ACC_{0-31} left OP_{25-31} places or right if OP-ve, OV' = 0
SHZ	CE	P	Shift while zero. Shift ACC left until $ACC_0 \neq 0$, OP' = no. of places, OV' = 0
AND	8A	P	And. ACC' = ACC & OP_{ACS}, OV' = 0
OR	3C	P	Or. ACC' = ACC v OP_{ACS}, OV' = 0
NEQ	8E	P	Not equivalent. ACC' = ACC ≠ OP_{ACS}, OV' = 0

DECIMAL

DAD	D0	P	Decimal add. ACC' = ACC + OP_{ACS}, OV' = overflow
DSB	D2	P	Decimal subtract. ACC' = ACC - OP_{ACS}, OV' = overflow
DRSB	D4	P	Decimal reverse subtract. ACC' = OP_{ACS} - ACC, OV' = overflow
DMY	DA	P	Decimal multiply. ACC' = ACC.OP_{ACS}, OV' = overflow
DMYD	DC	P	Decimal multiply double. ACS' = 2ACS, ACC' = ACC. OP_{ACS}, OV' = 0
DDV	9A	P	Decimal divide. ACC' = ACC/OP_{ACS}, OV' = 0
DMDV	9E	P	Decimal remainder divide. ACC' = ACC/OP_{ACS}, TOS_{ACS} = rem. OV' = 0, rem = 0 or rem > 0 & div > 0, CC' = 0, | rem > 0 & div < 0, CC' = 1, | rem < 0 & div > 0, CC' = 2, | rem < 0 & div < 0, CC' = 3 |
DRDV	9C	P	Decimal reverse divide. ACC' = OP_{ACS}/ACC, OV' = 0
DCP	D6	P	Decimal compare. ACC = OP_{ACS}, CC' = 0 | ACC < OP, CC = 1 | ACC > OP, CC = 2 |
DSH	D8	P	Decimal shift. Shift ACC left 4.OP places or right if -ve, OV' = overflow
CBIN	DE	P	Convert to binary. ACC' (binary) = ACC (pk decimal), OV' = overflow
CDEC	EE	P	Convert to decimal. ACS' = 2 ACS, ACC' (dec) = ACC (bin), OV' = 0

FIXED POINT (INTEGER)

IAD	EO	P	Add. ACC' = ACC + OP_{ACS}, OV' = overflow
ISB	E2	P	Subtract. ACC' = ACC-OP_{ACS}, OV' = overflow
IRSB	E4	P	Reverse subtract. ACC' = OP_{ACS} -ACC, OV' = overflow
IMY	EA	P	Multiply. ACC' = ACC.OP_{ACS}, OV' = overflow
IMYD	EC	P	Multiply double. ACS' = 2ACS, ACC' = ACC.OP_{ACS}, OV' = 0 ACC + ve, OP + ve, CC' = 0 | ACC + ve, OP - ve, CC' = 1, | ACC - ve, OP + ve, CC' = 2 | both - ve, CC = 3 |
IDV	AA	P	Divide. ACC' = ACC/OP_{ACS}, OV' = overflow
IRDV	AC	P	Reverse divide. ACC' = OP_{ACS}/ACC, OV' = overflow
IMDV	AE	P	Remainder divide. ACC' = ACC/OP_{ACS}, TOS' = rem, CC' as DMDV
ICP	E6	P	Compare. ACC = OP_{ACS}, CC' = 0, | ACC < OP, CC' = 1, | ACC > OP, CC' = 2 |
ISH	E8	P	Arithmetic shift. ACC' = ACC.2^i, i = OP_{26-31}. OV' = overflow i + ve, CC' = 3 | i - ve, all lost bits = 0, CC' = 0 | Last bit 0, CC' = 1 | Last bit 1, CC' = 2 |

FLOATING POINT (REAL)

RAD	F0	P	Floating add. ACC' = ACC + OP_{ACS}, OV' = overflow
RSB	F2	P	Floating subtract. ACC' = ACC - OP_{ACS}, OV' = overflow
RRSB	F4	P	Floating reverse subtract. ACC' = OP_{ACS} - ACC, OV' = overflow
RMY	FA	P	Floating multiply. ACC' = ACC.OP_{ACS}, OV' = overflow
RMYD	FC	P	Floating multiply double. ACS' = 2ACS, ACC = ACC.OP_{ACS}, OV' = overflow
RDV	BA	P	Floating divide. ACC' = ACC/OP_{ACS}, OV' = overflow
RRDV	BC	P	Floating reverse divide. ACC' = OP_{ACS}/ACC, OV' = overflow

FLOATING POINT (REAL) (Cont.)

RDVD	BE	P	Floating divide double. ACS' = ½ACS, ACC' = ACC/OP_{ACS}, OV' = overflow
RCP	F6	P	Floating compare. ACC = OP_{ACS}, CC' = 0 | ACC < OP, CC' = 1 | ACC > OP, CC = 2 |
RSC	F8	P	Scale. ACC' = ACC. 16^i. i = OP_{24-31}, OV' = overflow
FLT	A8	P	Float. ACS' = 2 ACS, ACC'_{FP} = ACC int . i; i = OP_{22-31} (biased) OV' = overflow
FIX	B8	P	Fix. ACC' int = ACC_{FP}, OP' = ACC_{exp} (unbiased), OV' = 0; if ACS = 128, ACS' = 64 and CC ≠ 0

EMULATION

COMA	98	P	Compress ACC. ACC' (6 bit chars) = ACC (8 bit bytes)
EXPA	88	P	Expand ACC. ACC' (8 bit bytes) = ACC (6 bit chars)

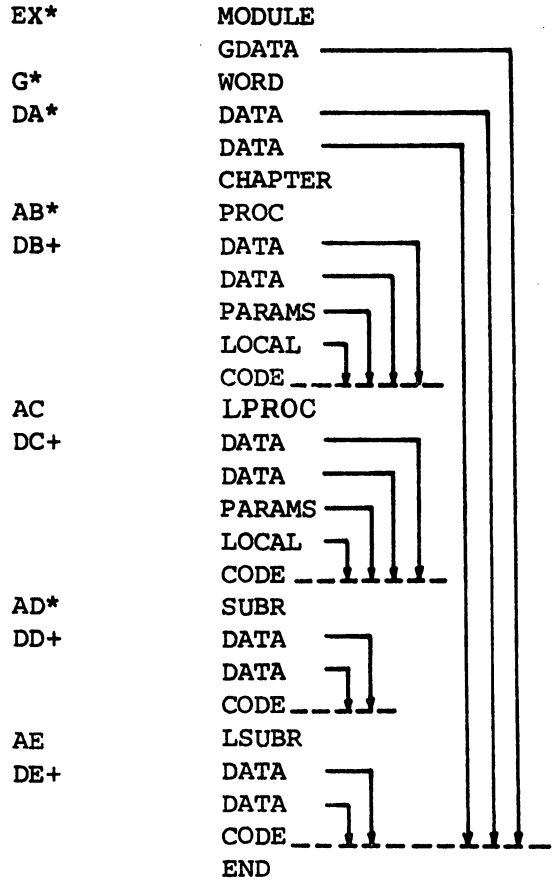
SFL Jumps

JE	Jump if equal	Compile into JCC
JNE	Jump if not equal	
JG	Jump if greater	
JGE	Jump if greater or equal	
JL	Jump if less	
JLE	Jump if less or equal	

Compile to JAT		Compile to JAF	
JN	Jump if negative	JNN	Jump if not negative
JND	Jump if negative, decimal	JNND	Jump if not negative, decimal
JNR	Jump if negative, real	JNNR	Jump if not negative, real
JNB	Jump if negative B	JNNB	Jump if not negative B
JP	Jump if positive	JNP	Jump if not positive
JPD	Jump if positive, decimal	JNPD	Jump if not positive, decimal
JPR	Jump if positive, real	JNPR	Jump if not positive, real
JPB	Jump if positive B	JNPB	Jump if not positive B
JZ	Jump if zero	JNZ	Jump if not zero
JZD	Jump if zero, decimal	JNZD	Jump if not zero, decimal
JZR	Jump if zero, real	JNZR	Jump if not zero, real
JZB	Jump if zero B	JNZB	Jump if not zero B
JZDL	Jump if zero, DR length	JNZDL	Jump if not zero, DR length
JOV	Jump if overflow	JNOV	Jump if not overflow

SCOPE OF DATA ITEMS

The following diagram shows the scope within which data items declared under the various classifiers can be used. Names marked * are visible to other SFL modules, and names marked + are visible to other routines within the module, by use of CDATA.



If you introduce items under data classifiers between sections of code, the effect is exactly as if you had declared the items before the code.

LITERAL FORMATS

Character	C' '
Hexadecimal	X' '
Binary	B'a,b-c'
Mask	M'000100...'
Repeated	Literal (number of repeats)
Multi-word	value/value/...

HEX DECODE TABLE

This table lists the 2900 collating sequence, hex values 00 to FF, giving the equivalent decimal values, basic functions, graphic characters, any special significance, paper tape codes and card codes. Graphic characters marked † are not part of the 64 character set.

EBCDIC Hexa-Decimal	Decimal	Function	Graphic Character	Special Significance	Tape Code	Card Code
00	0	Illegal		NULL	00/0	12-0-9-8-1
01	1			Start Of Heading (TC1)	00/1	12-9-1
02	2	JCC		Start of TeXt (TC2)	00/2	12-9-2
03	3			End of TeXt (TC3)	00/3	12-9-3
04	4	JAT			09/12	12-9-4
05	5			Horizontal Tabulate(FE1)	00/9	12-9-5
06	6	JAF			08/6	12-9-6
07	7			DELete	07/15	12-9-7
08	8				09/7	12-9-8
09	9				08/13	12-9-8-1
0A	10				08/14	12-9-8-2
0B	11			Vertical Tabulate (FE3)	00/11	12-9-8-3
0C	12			Form Feed (FE4)	00/12	12-9-8-4
0D	13			Carriage Return (FE5)	00/13	12-9-8-5
0E	14			Shift Out	00/14	12-9-8-6
0F	15			Shift In	00/15	12-9-8-7
10	16	VAL		Data Link Escape (TC7)	01/0	12-11-9-8-1
11	17			Device Control 1	01/1	11-9-1
12	18	CYD		Device Control 2	01/2	11-9-2
13	19			Device Control 3	01/3	11-9-3
14	20	INCA			09/13	11-9-4
15	21			New Line	08/5	11-9-5
16	22	MODD		Back Space (FE0)	00/8	11-9-6
17	23				08/7	11-9-7
18	24	PRCL		CANcel	01/8	11-9-8
19	25			End of Medium	01/9	11-9-8-1
1A	26	J			09/2	11-9-8-2
1B	27				08/15	11-9-8-3
1C	28	JLK		IS4 (File Separator)	01/12	11-9-8-4
1D	29			IS3 (Group Separator)	01/13	11-9-8-5
1E	30	CALL		IS2 (Record Separator)	01/14	11-9-8-6
1F	31			IS1 (Unit Separator)	01/15	11-9-8-7
20	32	ADB		Multiple SPace	08/0	11-0-9-8-1
21	33			Multiple New Line	08/1	0-9-1
22	34	SBB		Vertical Position	08/2	0-9-2
23	35				08/3	0-9-3
24	36	DEBJ			08/4	0-9-4
25	37			Line Feed (FE2)	00/10	0-9-5
26	38	CPB		End of Transmission Block (TC10)	01/7	0-9-6
27	39			ESCape	01/11	0-9-7
28	40	SIG			08/8	0-9-8
29	41				08/9	0-9-8-1
2A	42	MYB			08/10	0-9-8-2
2B	43				08/11	0-9-8-3
2C	44	VMY			08/12	0-9-8-4
2D	45			ENQuiry (TC5)	00/5	0-9-8-5
2E	46	CPIB		ACKnowledge (TC6)	00/6	0-9-8-6
2F	47			BELL	00/7	0-9-8-7
30	48	LCT			09/0	12-11-0-9-8-1
31	49				09/1	9-1
32	50	MPSR		SYNchronous idle (TC9)	01/6	9-2
33	51				09/3	9-3
34	52	CPSR			09/4	9-4
35	53				09/5	9-5
36	54	STCT			09/6	9-6
37	55			End of Transmission (TC4)	00/4	9-7
38	56	EXIT			09/8	9-8
39	57				09/9	9-8-1
3A	58	ESEX			09/10	9-8-2
3B	59				09/11	9-8-3
3C	60	OUT		Device Control 4	01/4	9-8-4
3D	61			Negative acKnowledge (TC8)	01/5	9-8-5
3E	62	ACT			09/14	9-8-6
3F	63			SUBstitute	01/10	9-8-7
40	64	SL	Δ	SPace	02/0	NO PUNCH
41	65				10/0	12-0-9-1
42	66	SLSS			10/1	12-0-9-2
43	67				10/2	12-0-9-3
44	68	SLSD			10/3	12-0-9-4
45	69				10/4	12-0-9-5

EBCDIC Hexa-Decimal	Decimal	Function	Graphic Character	Special Significance	Tape Code	Card Code
46	70	SLSQ			10/5	12-0-9-6
47	71				10/6	12-0-9-7
48	72	ST			10/7	12-0-9-8
49	73				10/8	12-8-1
4A	74	STUH	[		05/11	12-8-2
4B	75		.		02/14	12-8-3
4C	76	STXN	<		03/12	12-8-4
4D	77		(		02/8	12-8-5
4E	78	IDLE	+		02/11	12-8-6
4F	79		|		02/1	12-8-7
50	80	SLD	&		02/6	12
51	81				10/9	12-11-9-1
52	82	SLB			10/10	12-11-9-2
53	83				10/11	12-11-9-3
54	84	TDEC			10/12	12-11-9-4
55	85				10/13	12-11-9-5
56	86	INCT			10/14	12-11-9-6
57	87				10/15	12-11-9-7
58	88	STD			11/0	12-11-9-8
59	89				11/1	11-8-1
5A	90	STB	]		05/13	11-8-2
5B	91		$		02/4	11-8-3
5C	92	STLN	*		02/10	11-8-4
5D	93		)		02/9	11-8-5
5E	94	STSF	;		03/11	11-8-6
5F	95		^		05/14	11-8-7
60	96	L	–		02/13	11
61	97		/		02/15	0-1
62	98	LSS			11/2	11-0-9-2
63	99				11/3	11-0-9-3
64	100	LSD			11/4	11-0-9-4
65	101				11/5	11-0-9-5
66	102	LSQ			11/6	11-0-9-6
67	103				11/7	11-0-9-7
68	104	RRTC			11/8	11-0-9-8
69	105		|		11/9	0-8-1
6A	106	LUH	¦ †		07/12	12-11
6B	107		,		02/12	0-8-3
6C	108	RALN	%		02/5	0-8-4
6D	109		—		05/15	0-8-5
6E	110	ASF			03/14	0-8-6
6F	111		?		03/15	0-8-7
70	112	LDRL			11/10	12-11-0
71	113				11/11	12-11-0-9-1
72	114	LDA			11/12	12-11-0-9-2
73	115				11/13	12-11-0-9-3
74	116	LDTB			11/14	12-11-0-9-4
75	117				11/15	12-11-0-9-5
76	118	LDB			12/0	12-11-0-9-6
77	119				12/1	12-11-0-9-7
78	120	LD			12/2	12-11-0-9-8
79	121		` †		06/0	8-1
7A	122	LB	:		03/10	8-2
7B	123		£		02/3	8-3
7C	124	LLN	@		04/0	8-4
7D	125		'		02/7	8-5
7E	126	LXN	=		03/13	8-6
7F	127		"		02/2	8-7
80	128	TCH			12/3	12-0-8-1
81	129		a †		06/1	12-0-1
82	130	ANDS	b †		06/2	12-0-2
83	131		c †		06/3	12-0-3
84	132	ORS	d †		06/4	12-0-4
85	133		e †		06/5	12-0-5
86	134	NEQS	f †		06/6	12-0-6
87	135		g †		06/7	12-0-7
88	136	EXPA	h †		06/8	12-0-8
89	137		i †		06/9	12-0-9
8A	138	AND			12/4	12-0-8-2
8B	139				12/5	12-0-8-3
8C	140	OR			12/6	12-0-8-4
8D	141				12/7	12-0-8-5
8E	142	NEQ			12/8	12-0-8-6
8F	143				12/9	12-0-8-7
90	144	PK			12/10	12-11-8-1
91	145		j †		06/10	12-11-1
92	146	INS	k †		06/11	12-11-2
93	147		l †		06/12	12-11-3
94	148	SUPK	m †		06/13	12-11-4
95	149		n †		06/14	12-11-5

EBCDIC Hexa-Decimal	Decimal	Function	Graphic Character	Special Significance	Tape Code	Card Code
96	150		o †		06/15	12-11-6
97	151		p †		07/0	12-11-7
98	152	COMA	q †		07/1	12-11-8
99	153		r †		07/2	12-11-9
9A	154	DDV			12/11	12-11-8-2
9B	155				12/12	12-11-8-3
9C	156	DRDV			12/13	12-11-8-4
9D	157				12/14	12-11-8-5
9E	158	DMDV			12/15	12-11-8-6
9F	159				13/0	12-11-8-7
A0	160	SWEQ			13/1	11-0-8-1
A1	161		~ †		07/14	11-0-1
A2	162	SWNE	s †		07/3	11-0-2
A3	163		t †		07/4	11-0-3
A4	164	CPS	u †		07/5	11-0-4
A5	165		v †		07/6	11-0-5
A6	166	TTR	w †		07/7	11-0-6
A7	167		x †		07/8	11-0-7
A8	168	FLT	y †		07/9	11-0-8
A9	169		z †		07/10	11-0-9
AA	170	IDV			13/2	11-0-8-2
AB	171				13/3	11-0-8-3
AC	172	IRDV			13/4	11-0-8-4
AD	173				13/5	11-0-8-5
AE	174	IMDV			13/6	11-0-8-6
AF	175				13/7	11-0-8-7
B0	176	MVL			13/8	12-11-0-8-1
B1	177				13/9	12-11-0-1
B2	178	MV			13/10	12-11-0-2
B3	179				13/11	12-11-0-3
B4	180	CHOV			13/12	12-11-0-4
B5	181				13/13	12-11-0-5
B6	182				13/14	12-11-0-6
B7	183				13/15	12-11-0-7
B8	184	FIX			14/0	12-11-0-8
B9	185				14/1	12-11-0-9
BA	186	RDV			14/2	12-11-0-8-2
BB	187				14/3	12-11-0-8-3
BC	188	RRDV			14/4	12-11-0-8-4
BD	189				14/5	12-11-0-8-5
BE	190	RDVD			14/6	12-11-0-8-6
BF	191				14/7	12-11-0-8-7
C0	192	UAD	{ †		07/11	12-0
C1	193		A		04/1	12-1
C2	194	USB	B		04/2	12-2
C3	195		C		04/3	12-3
C4	196	URSB	D		04/4	12-4
C5	197		E		04/5	12-5
C6	198	UCP	F		04/6	12-6
C7	199		G		04/7	12-7
C8	200	USH	H		04/8	12-8
C9	201		I		04/9	12-9
CA	202	ROT			14/8	12-0-9-8-2
CB	203				14/9	12-0-9-8-3
CC	204	SHS		Prefix code	14/10	12-0-9-8-4
CD	205				14/11	12-0-9-8-5
CE	206	SHZ			14/12	12-0-9-8-6
CF	207				14/13	12-0-9-8-7
D0	208	DAD	} †		07/3	11-0
D1	209		J		04/10	11-1
D2	210	DSB	K		04/11	11-2
D3	211		L		04/12	11-3
D4	212	DRSB	M		04/13	11-4
D5	213		N		04/14	11-5
D6	214	DCP	O		04/15	11-6
D7	215		P		05/0	11-7
D8	216	DSH	Q		05/1	11-8
D9	217		R		05/2	11-9
DA	218	DMY			14/14	12-11-9-8-2
DB	219				14/15	12-11-9-8-3
DC	220	DMYD			15/0	12-11-9-8-4
DD	221				15/1	12-11-9-8-5
DE	222	CBIN			15/2	12-11-9-8-6
DF	223				15/3	12-11-9-8-7
E0	224	IAD	\		05/12	0-8-2
E1	225				09/15	11-0-9-1
E2	226	ISB	S		05/3	0-2
E3	227		T		05/4	0-3
E4	228	IRSB	U		05/5	0-4
E5	229		V		05/6	0-5
E6	230	ICP	W		05/7	0-6

EBCDIC Hexa-Decimal	Decimal	Function	Graphic Character	Special Significance	Tape Code	Card Code
E7	231		X		05/8	0-7
E8	232	ISH	Y		05/9	0-8
E9	233		Z		05/10	0-9
EA	234	IMY			15/4	11-0-9-8-2
EB	235				15/5	11-0-9-8-3
EC	236	IMYD			15/6	11-0-9-8-4
ED	237				15/7	11-0-9-8-5
EE	238	CDEC			15/8	11-0-9-8-6
EF	239				15/9	11-0-9-8-7
F0	240	RAD	0		03/0	0
F1	241		1		03/1	1
F2	242	RSB	2		03/2	2
F3	243		3		03/3	3
F4	244	RRSB	4		03/4	4
F5	245		5		03/5	5
F6	246	RCP	6		03/6	6
F7	247		7		03/7	7
F8	248	RSC	8		03/8	8
F9	249		9		03/9	9
FA	250	RMY			15/10	12-11-0-9-8-2
FB	251				15/11	12-11-0-9-8-3
FC	252	RMYD			15/12	12-11-0-9-8-4
FD	253				15/13	12-11-0-9-8-5
FE	254				15/14	12-11-0-9-8-6
FF	255	Illegal		Eight ones	15/15	12-11-0-9-8-7